UNIVERSIDAD
DE MÁLAGA

Sistemas de Tiempo Real

Seminario



Cipriano Galindo

DEPARTAMENTO DE INGENIERÍA
DE SISTEMAS Y AUTOMÁTICA

FreeRTOS

- Pequeño sistema operativo que puede funcionar en casi cualquier microcontrolador.
  - Bajo consumo de memoria
  - Bajo consumo CPU
  - Sólo requiere de un temporizador (TIMER)
  - Diseñado para sistemas de tiempo real

- Ejecución concurrente de tareas
  - Mayor flexibilidad para el programador
  - Prioridad de ejecución
  - Semáforos
  - Colas de mensajes
  - …

FreeRTOS

- Configurable para que se adapte a diferentes plataformas.
- En el fichero FreeRTOSConfig.h, por ejemplo:

```
                                        …
#define configUSE_PREEMPTION            1
#define configUSE_IDLE_HOOK             1
#define configUSE_TICK_HOOK             0
#define configCPU_CLOCK_HZ              ( ( unsigned long ) 16000000)
#define configTICK_RATE_HZ              ( ( portTickType ) 1000 )
#define configMAX_PRIORITIES            ( ( unsigned portBASE_TYPE ) 4 )
#define configMINIMAL_STACK_SIZE        ( ( unsigned short ) 85 )
#define configTOTAL_HEAP_SIZE           ( (size_t ) ( 1500 ) )
#define configMAX_TASK_NAME_LEN                 ( 8 )
#define configUSE_TRACE_FACILITY        0
                                        …
```

- Se ha modificado para utilizar el **timer 1** por el repartidor de tareas: cada tick de reloj (1 ms) comprueba qué tarea debe estar siendo ejecutada
- Las tareas se han modificado para que dejen una traza en la memoria RAM de la placa de desarrollo para trazar su ejecución.

FreeRTOS

- Creación de tareas:

```
#include “task.h”

xReturned = xTaskCreate(
                tskFunction,       /* Function that implements the task. */
                "NAME",           /* Text name for the task. */
                STACK_SIZE,       /* Stack size in words, not bytes. */
                ( void * ) 1,     /* Parameter passed into the task. */
                tskIDLE_PRIORITY,/* Priority of the task. */
                &xHandle );       /* handle to the task. */
```

- Función con el código de la tarea. Será el repartidor quien se encargue de llamarla, interrumpirla, y reanudarla.

```
void vTaskCode( void * pvParameters )
{

        for( ;; )
            {
                /* Task code goes here. */
            }
}
```

FreeRTOS

- Se inicia el repartidor llamando a la función:

```
vTaskStartScheduler();
```

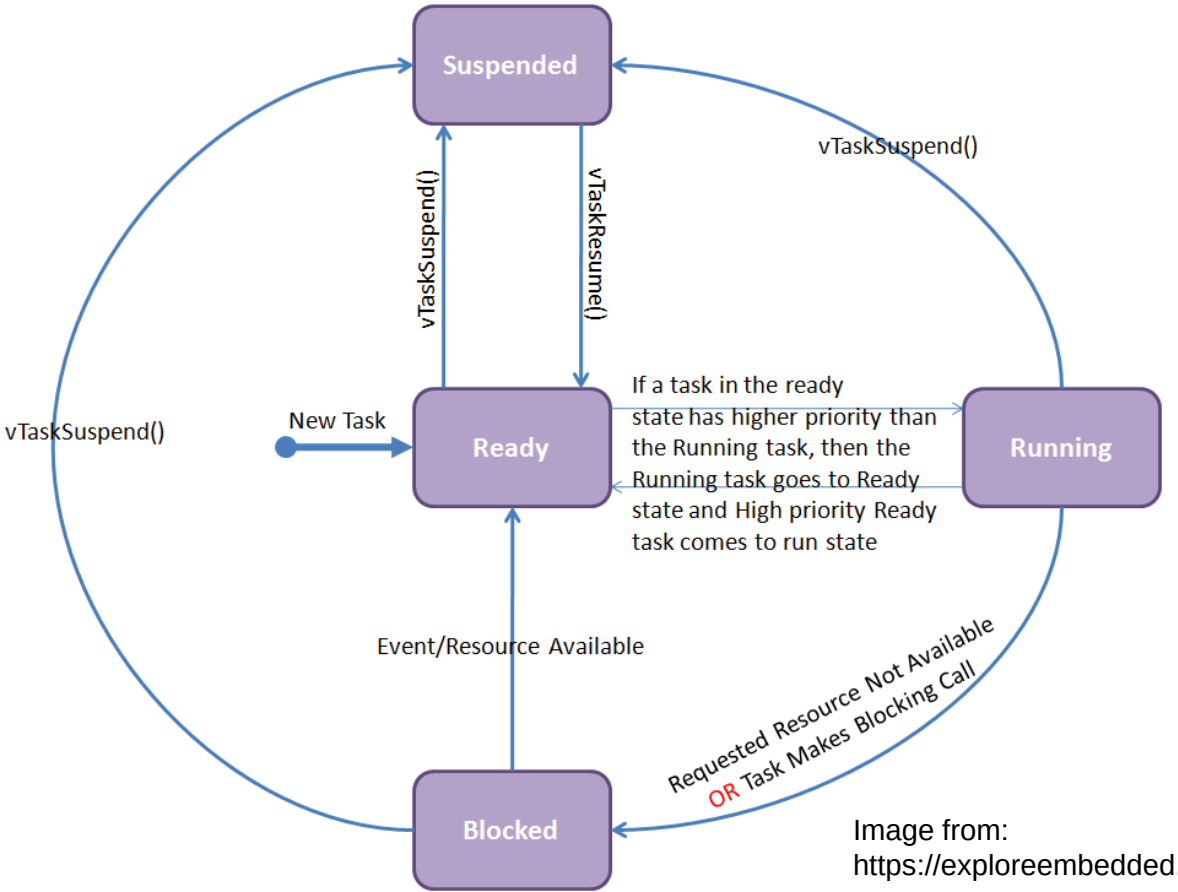
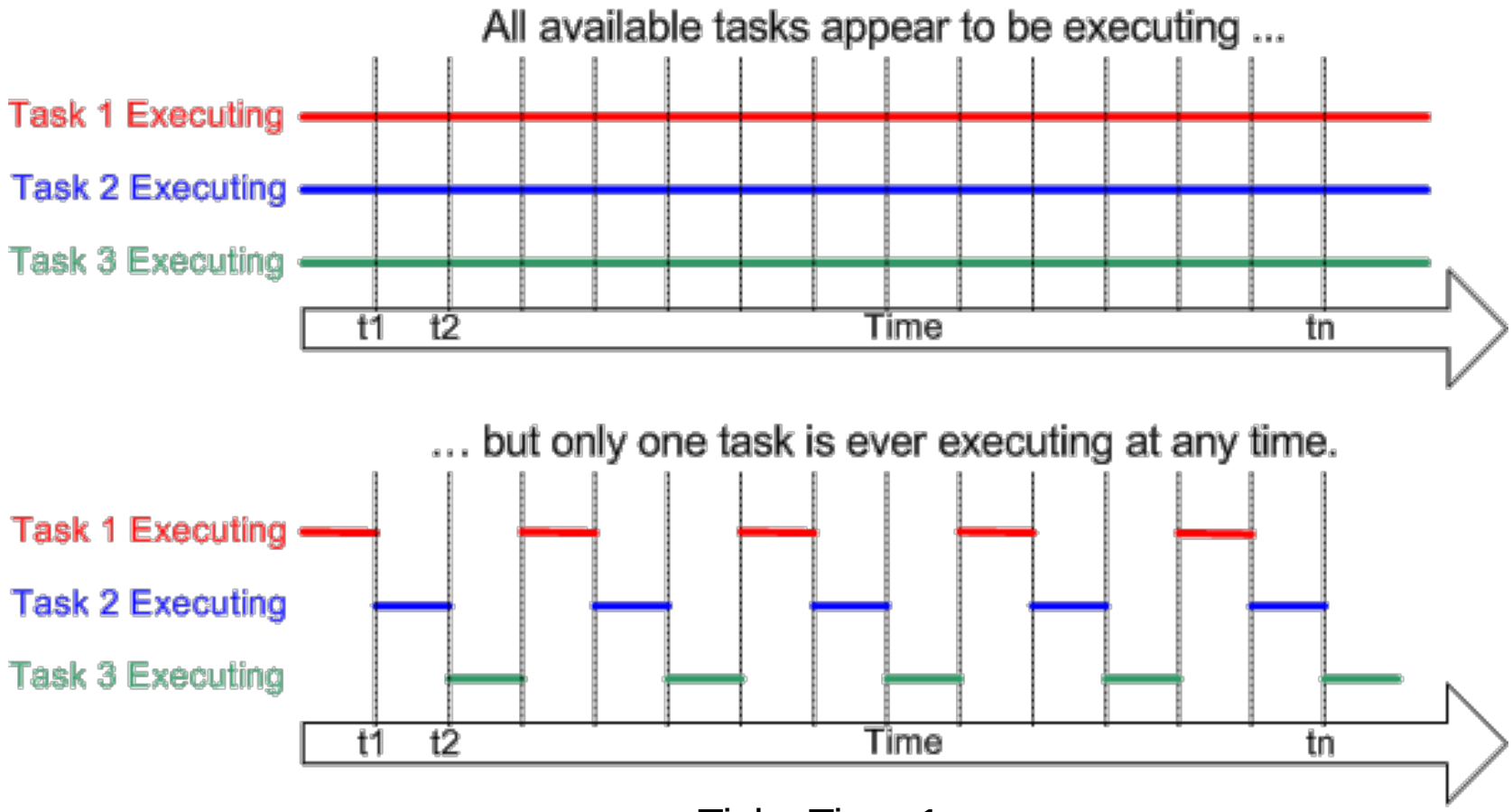


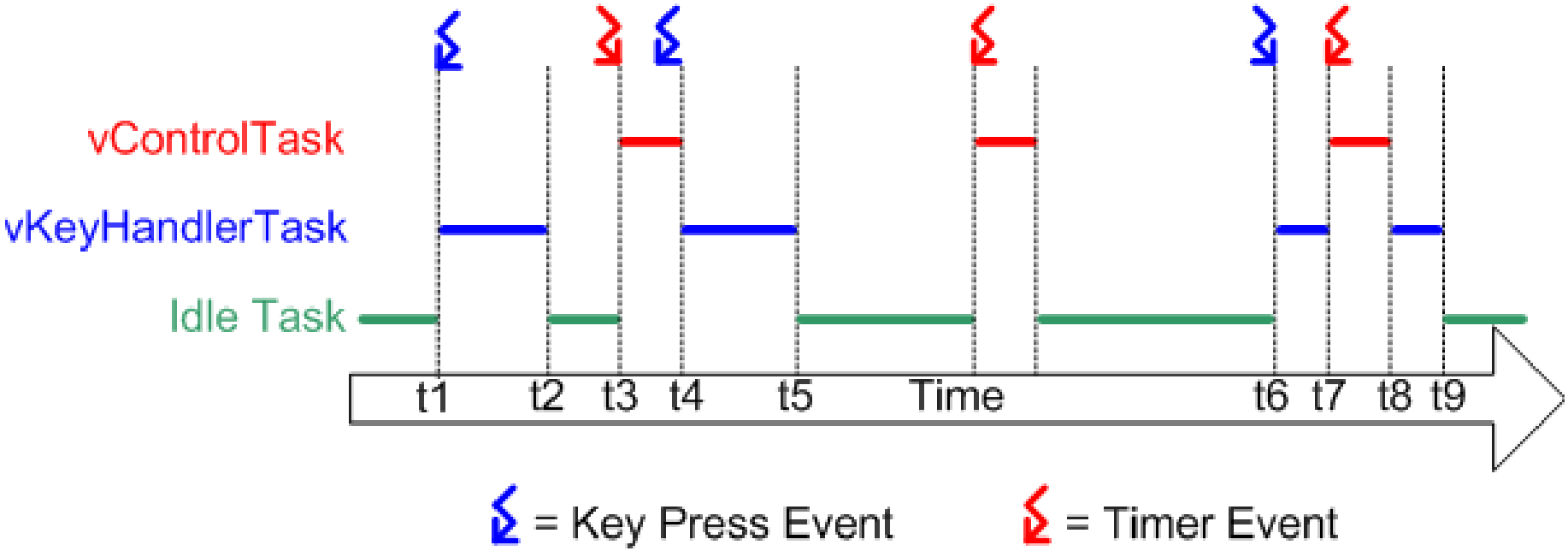
Image from:
https://exploreembedded.com/wiki/Task_Switching

Tareas Concurrentes



Ticks Timer1

Tareas Concurrentes



Prioridades

- Las tareas con mayor prioridad tienen preferencia sobre las de menor
  - Pueden “quitar” CPU
  - Pero deben respetar secciones críticas de código (recursos bloqueantes).
- A igual prioridad, las tareas se turnan.

- Si no hay ningún tarea deseando ser ejecutada, se ejecuta “idle_task” por defecto.

Semáforo binario

- Actúa como una “llave” que permite acceso a un recurso. Si una tarea tiene la llave, los otros no pueden cogerla.
- Es responsabilidad del programador implementar un código que respete el semáforo.
- Si el programador no lo respeta, el código **puede** entrar en la sección critica ignorando las protecciones y corromper los datos de otras tareas.

```
#include “semphr.h”

SemaphoreHandle_t xSemaphoreCreateBinary( void );
```

Se crea en estado “vacío”. Así que hay que habilitarlo:

```
xSemaphoreGive(SemaphoreHandle_t xSemaphore)
```

Antes de tomarlo:

```
xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                TickType_t xTicksToWait )
```

Mutex

Un mutex es un semáforo binario que **implementa herencia de prioridades**, por lo que está exento del problema de la inversión de prioridades.

```
SemaphoreHandle_t xSemaphoreCreateMutex( void )
```

Utilizan las mismas funciones que los semáforos binarios para las operaciones de "give" y "take"

Counting semaphores

Son semáforos que pueden tener más de una "llave", es decir, permiten ser accedidos más de una vez.

```
SemaphoreHandle_t xSemaphoreCreateCounting (
                UBaseType_t uxMaxCount,
                UBaseType_t uxInitialCount);
```

Cada vez que se realiza la operación de "give" se incrementa la cuenta en uno y cada vez que se hace "take" se decrementa. El semáforo está bloqueado cuando